

2. $\{0,1\}^*$

i) All the strings beginning and ending with 1;

Regular Expression $= 1(0+1)^*1$

ii) All the string containing the substring 01.

Regular Expression $= (0+1)^*01(0+1)^*$

iii) All the string with exactly one 0.

Regular Expression $= 1^*01^*$

iv) All the strings with at least two 1s

Regular Expression $= (0+1)^*1(0+1)^*1(0+1)^*$

v) All the strings that do not end with 0

Regular Expression $= 1+(0+1)^*1$

3. i) $(\epsilon + aa^*)^*$

$$= (a^*)^* \quad [\because \epsilon + aa^* = (a^*)^* = a^*]$$

$$= a^* \quad (\because (a^*)^* = a^*)$$

ii) $(a^* + b^*)^* abba$

$$(a^* + b^*)^* abba = (a+b)^* abba$$

$$L = abba$$

iii) $(a^*b^*)^* aba$

$$(a^*b^*)^* = (a+b)^*$$

$$= (a+b)^* aba$$

$$L = aba$$

$$iv) (a+ab)^*$$

$$L = \{\epsilon, aa, ab, aab, aba, \dots\}$$

$$v) (aba)^* + (a^*b^*)^*$$

$$(a^*b^*)^* = (a+b)^*$$

$$(aba)^* = \{\epsilon, aba, abaaba, \dots\}$$

$$5) i) (0^*1^*)^* = (0+1)^*$$

$$(0^*1^*)^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

$$(0+1)^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

$$(0^*1^*)^* = (0+1)^*$$

$$ii) (0+1)^* (0+1)^* = (0+1)^*$$

$$\tilde{R} R^* = R^*$$

$$iii) (0^*)^* = 0^*$$

$$(R^*)^* = R^*$$

$$iv) (0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$$

~~$$(0+1)^* 0 (0+1)^*$$~~

~~$$(0+1)^* 0 + (0+1)^* 1 = (0+1)^*$$~~

$$(0+1)^* (1+0) = (0+1)^*$$

$$(0+1)^* = (0+1)^*$$

$$L.H.S = R.H.S$$

$$v) \epsilon + 0(0+1)^* = 0^*$$

$$\epsilon + 0R(R^*) = R^*$$

4.1.1

```
#include <stdio.h>
```

```
+
```

```
+
```

```
if 1
```

```
else 1
```

```
while 1
```

```
for 1
```

```
printf("keyword\n");
```

```
[A-z a-z]([A-z a-z]) {printf("Identifier ");
```

```
[0-9] {printf("consonant ");
```

```
"/" {printf("operators: ");
```

```
"/" {printf("delimiter: ");
```

```
[ \t\n] +;
```

```
+
```

```
int main()
```

```
{
```

```
printf("Enter the text:\n");
```

```
getline();
```

```
}
```

```
int yywrap() {
```

```
return 1;
```

```
}
```

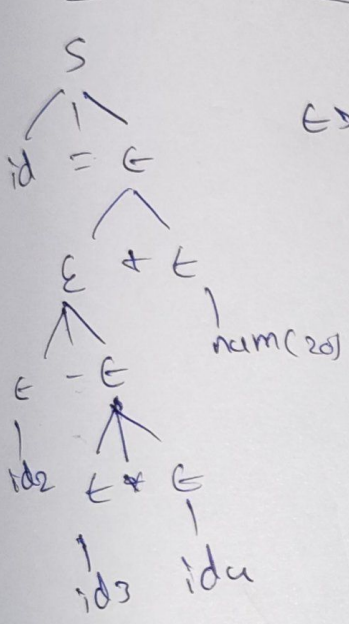
$$w = n * y - z + 20$$

where: n, y, z, w are of type real

i) Lexical Analysis

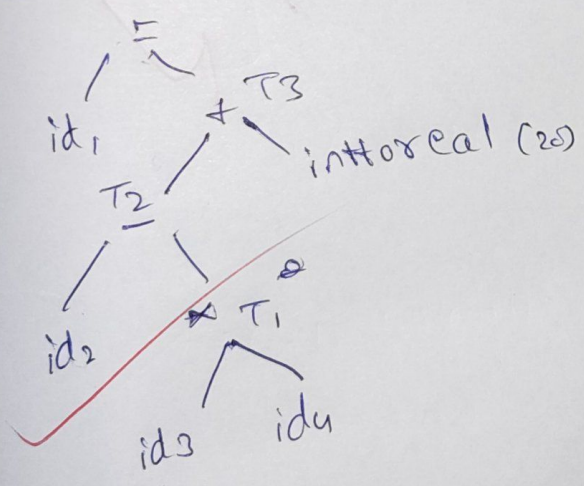
$\langle id \rangle \langle = \rangle \langle + \rangle \langle - \rangle \langle * \rangle \langle / \rangle \langle \text{num} \rangle \langle \text{int} \rangle \langle \text{real} \rangle$

ii) Syntax Analysis:-



$S \Rightarrow id = E$
 $E \Rightarrow E + E \mid E - E \mid E * E \mid id \mid num$

iii) Semantic Analysis



iv) ZLG:-

$$T_3 = \text{intoReal}(20)$$

$$T_4 = T_2 + T_3$$

$$id_1 = T_4$$

v) code optimizer

$$T_1 = id_3 + id_4$$

$$T_2 = id_2 - T_1$$

$$id_1 = T_2 + 20.0$$

~~vi) code~~
mo

vi) MOV id_3, R_3

MOV id_4, R_2

MUL R_3, R_2

MOV id_2, R_1

SUB R_2, R_1

~~ADD~~ ADD $\#20.0, R_1$

MOV R_1, id_1

